

# AI Platform Developer — Take-Home Assessment

## Contract Intelligence Pipeline

---

### Welcome

Thank you for your interest in the AI Platform Developer role. This take-home assessment is designed to give you an opportunity to demonstrate how you think about building production-grade AI systems — not just whether you can make something work, but whether you can make it *last*.

You'll work through a realistic scenario in three phases: data ingestion, multi-agent orchestration, and production documentation. The scenario is drawn from real challenges in architecture, engineering, and construction (AEC) — an industry where contracts, compliance, and data integrity are critical to every project.

### Important things to know before you start:

- **Time commitment:** 90–120 minutes. Please do not spend more than two hours.
  - **We value simplicity and clarity over architectural ambition.** Clean, readable code with thoughtful documentation will always outscore a complex system that's hard to follow.
  - **Partial solutions with clear documentation of what you'd do next score higher than rushed completeness.** If you run out of time, tell us what's left and how you'd approach it.
  - **Framework choice is entirely yours.** LangChain, CrewAI, AutoGen, raw Python with API calls — all are acceptable. We're testing your thinking, not your tool preferences.
  - **You may mock or stub LLM calls.** If you do, make it clear where real calls would go and what the interface contract looks like. The orchestration, logging, and control surfaces must be real.
  - **We're looking for production thinking.** How you approach durability, governance, and operability matters as much as whether the code runs.
- 

### The Scenario

An AEC firm is building a **Contract Intelligence Pipeline** on a governed data platform that uses a medallion architecture (Bronze → Silver → Gold). Contract clause data arrives from an external system via API. The firm needs to:

1. Ingest the data reliably into a Bronze layer
2. Use AI agents to classify clauses, flag risks, and log decisions

3. Ensure the system is operable, auditable, and maintainable by the team

You've been asked to build a working proof-of-concept that demonstrates production thinking — not just functionality, but durability.

---

## What You've Been Given

| File                                        | Description                                                                                                                      |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <code>clauses_batch_1.json</code>           | First batch of contract clause data from the API (original schema)                                                               |
| <code>clauses_batch_2.json</code>           | Second batch — same source, but the schema has changed (a field was renamed, a new nested object appeared, a column was removed) |
| <code>clauses_ingested_fallback.json</code> | Pre-ingested dataset you can use for Phase 2 if you need to move on from Phase 1                                                 |
| This document                               | Scenario brief and phase instructions                                                                                            |

---

## Phase 1 — Ingest & Handle the Unexpected (25–30 minutes)

**Your task:** Write a Python ingestion script that lands the contract clause data into a Bronze-layer structure.

### Requirements:

1. **Land data into a Bronze-layer structure.** Partition by ingestion timestamp. Preserve raw data as-is — no transformation at this stage.
2. **Detect and surface the schema drift.** When `clauses_batch_2.json` arrives with a different schema than batch 1, your system must detect this and make it visible (log it, raise it, write it to a drift report — your choice). It must **not** silently drop or ignore changed data.
3. **Implement basic resumability.** If the script fails mid-file (e.g., after processing 50 of 100 records), it should be able to pick up where it left off without reprocessing completed records.
4. **Define the data contract.** Write a brief specification (inline comments, a separate short doc, or a schema file) that tells downstream consumers: what fields to expect, what guarantees you make, and what known limitations exist.

### Scope guardrails:

- This is not a full pipeline framework. Keep it simple and legible.

- We are not testing distributed compute or Spark. Plain Python, local file I/O, and clear logic are perfectly appropriate.
- Focus on correctness and resilience over abstraction.

**Output:** Python script(s) + data contract specification + any output files your script produces.

---

## Phase 2 — Multi-Agent Contract Review with Guardrails (40–50 minutes)

**Your task:** Build a working multi-agent system that processes ingested contract clauses.

Use the output from your Phase 1 ingestion, **or** use the provided `clauses_ingested_fallback.json` if you'd prefer to move directly to this phase. Either approach is completely fine.

### Build 2–3 agents:

**Agent 1 — Clause Classifier** Categorizes each clause by type: liability, insurance, indemnification, payment terms, termination, scope of work, or other. Outputs a structured classification with confidence level.

**Agent 2 — Risk Flagging Agent** Reviews classified clauses and identifies unfavorable language. For each flag: states what the risk is, suggests an alternative, and logs a confidence score.

**Agent 3 — Audit & Learning Agent** Logs every decision made by Agents 1 and 2 with structured metadata. Records when a human overrides a suggestion (you can simulate this). Describes how the system would incorporate corrections over time.

### Required implementation elements:

**Structured logging (required):** Every agent action must produce a log entry containing *at minimum*:

- Timestamp
- Agent name
- Input reference (clause ID)
- Action taken
- Output / decision
- Clear separation between model output and human decision

**At least one guardrail (required):** Implement at least one of the following:

- Token or cost budget limit (processing stops when budget is reached)
- Scope boundary (agent refuses or escalates out-of-scope requests)
- Kill-switch mechanism (halts all processing on a defined condition)

**Human-in-the-loop boundary (required — this is a pass/fail element):** The system must **not** automatically modify its own behavior, prompts, or decision logic without an explicit human approval step. The "learning" component should describe the mechanism for incorporating feedback — it must not self-execute that learning. This constraint is intentional and reflects how production AI systems must operate in regulated and high-stakes environments.

**Scope guardrails:**

- Mocked or stubbed LLM responses are acceptable. The orchestration, logging, guardrails, and control surfaces must be real.
- If you mock LLM calls, make it clear where real calls would be integrated and what the interface looks like.
- You do not need to implement actual NLP/ML classification. Simulated outputs with realistic structure are fine.

**Output:** Python code for the agent system + structured log output + any configuration or contract files.

---

### **Phase 3 — Make It Durable (20–25 minutes)**

**Your task:** Produce the documentation that makes this system operable by someone who didn't build it.

**README (required)** A teammate who was not involved in building this should be able to:

- Understand the architecture (what runs, in what order, and why)
- Run the system (setup, dependencies, execution steps)
- Know where to extend it (what's modular, what's hardcoded, what needs work)
- Understand your assumptions and known limitations

**Architecture Decision Note (1 page max)** Explain your key design decisions and tradeoffs. What did you prioritize and why? What would you change with more time, more data, or a production deployment?

**Runbook Excerpt (required)** Cover three operational scenarios:

1. **Monitoring:** How do you know the system is healthy? What do you watch? What's normal vs. concerning?
2. **Agent failure:** An agent fails mid-workflow. Who gets alerted? How do you recover? What happens to in-flight data?
3. **Rollback:** Something went wrong in production. How do you revert to a previous known-good state?

**Buy vs. Build Rationale (one component only)** Pick one component of your solution. Write a brief rationale for why you built it vs. used an existing tool (or vice versa). Include:

- When you would revisit that decision
- What signal would tell you it's time to sunset or replace it

**Scope guardrails:**

- These should be concise and practical, not polished essays.
- If you documented as you built (inline comments, commit messages, etc.), reference that rather than rewriting.
- Bullet points and short paragraphs are preferred over long prose.

**Output:** README + Architecture note + Runbook excerpt + Buy vs. build rationale.

---

## **Submission Instructions**

**What to submit:** A single GitHub repository (preferred) or zip file containing:

- All Python source code
- All output files your code produces (logs, Bronze-layer output, drift reports, etc.)
- Data contract specification (from Phase 1)
- README, architecture note, runbook excerpt, and buy-vs-build rationale (from Phase 3)
- The provided data files (so reviewers can run your code)

**Structure suggestion (not required, but helpful):**

```
|— README.md
|— data/
|   |— clauses_batch_1.json
|   |— clauses_batch_2.json
|   └— clauses_ingested_fallback.json
|— phase1_ingestion/
|   |— ingest.py
|   |— data_contract.md (or .json)
|   └— output/
|— phase2_agents/
|   |— agents.py (or split by agent)
|   |— config/
|   └— output/
|       └— audit_log.json
|— docs/
|   |— architecture_note.md
|   |— runbook.md
|   └— buy_vs_build.md
```

**Questions:** If anything in this brief is unclear, please reach out. Asking clarifying questions is encouraged and will not count against you.